InventIQ – Reparto de la Entrega 2

Oct 16, 2026 · @InvetIQ

Resumen

El reparto por letras del cronograma de Sebas se mantiene, con tres ajustes: se reequilibra la carga de A, se recorren las fechas porque el arranque ya va unos tres días tarde, y se agregan entregables que pide la Entrega 2 y faltaban.

- **Carga:** A tenía 14 tareas frente a 7–9 de las demás. Los repositorios pasan a quien los usa, los datos de demostración pasan a C y Twilio sale de la Entrega 2.
- **Fechas:** el esqueleto y el modelo (A1–A2) estaban para el 13–15 de octubre y hoy, viernes 16, apenas empieza la base. Todo se corre lo justo para no tocar la fecha de entrega del jueves 5 de noviembre.
- **Faltantes:** el documento de arquitectura, la declaración de uso de IA en el README y el umbral de cobertura del 80% desde ya.

Cada parte queda con su lista completa de tareas, ordenada por fecha. A queda asignada a Alejandro porque ya está montando la base; B, C y D quedan para que los demás elijan.

Revisión del cronograma

El cronograma es sólido: el alcance de la Entrega 2 (login, venta FIFO, ROP, borradores y recepción) cumple de sobra con "al menos una regla de punta a punta", y las dependencias están bien identificadas. Estos son los puntos que conviene corregir.

#	Hallazgo	Ajuste propuesto
1	A2 (modelo e interfaces) bloquea a las cuatro letras y ya va atrasada.	Reunión corta para cerrar el modelo antes de programarlo. Mientras A lo fusiona, B y C escriben sus reglas puras con pruebas, que no dependen de Spring.
2	A concentra 14 tareas, incluidos los repositorios de todas las entidades (A6, A7) y los datos semilla (A8). Es el cuello de botella.	Cada letra hace los repositorios de sus entidades. C hace los datos de demostración porque sabe qué historial necesita el ROP.

#	Hallazgo	Ajuste propuesto
3	Twilio (A5, A10) está en la Entrega 2, pero FR-09 es de la Entrega 3. Además, sus credenciales exigen variables de entorno, y el curso pide arrancar sin configurar nada.	En la Entrega 2 solo va un NotificadorMensajes falso que escribe en el log. El adaptador real se activa en la Entrega 3 únicamente si hay credenciales.
4	No aparece el documento de arquitectura (capas, modelo de dominio, dependencias), que la Entrega 2 exige explícitamente.	Se agrega a A, con una sección por letra.
5	La regla del 80% de cobertura solo se activa en la Entrega 3 (A24).	Umbral de JaCoCo sobre el paquete de dominio desde A3: el build falla si baja de 80%.
6	Faltan la sección de uso de IA en el README y la justificación escrita de la API externa.	El README de uso de IA va en la Entrega 2. La justificación de Twilio puede esperar a la Entrega 3, cuando se integre.
7	En la defensa pueden preguntarle a cualquiera por código que no escribió.	Cada PR lo aprueba una letra distinta cada vez, y se hace un ensayo cruzado de preguntas el 4 de noviembre.

Preguntas abiertas:

- Confirmar con la profesora la fecha exacta de la Entrega 2 y si pide algo además del código y el documento.
- Este documento no incluye ACUERDOS.md; las referencias a §3–§6 se toman del cronograma tal cual.

Calendario ajustado

La fecha de entrega se mantiene en el jueves 5 de noviembre. Lo que se corre son los hitos intermedios, y lo que se sacrifica es parte del colchón de la semana 10.

1. **Sáb 17 oct:** reunión de 1 hora para cerrar el modelo de dominio y las interfaces, y poner nombre a las letras B, C y D. Esqueleto (A1) fusionado.
2. **Mar 20 oct:** modelo e interfaces (A2) fusionados, CI y README v1 listos. Los cuatro arrancan el proyecto con un solo comando.
3. **Lun 26 oct:** demo interna 1 (antes vie 23). Cada módulo se muestra por separado.
4. **Jue 29 oct:** día de integración (antes mié 28). El ciclo de login, venta, ROP, borrador, aprobación y recepción corre sin tocar la BD a mano.

5. **Sáb 31 oct:** congelamiento y etiqueta v0.2. Desde aquí solo se corrigen errores.
6. **Mar 3 nov:** secciones del documento y documento de arquitectura completos (lun 2 es festivo).
7. **Mié 4 nov:** ensayo de la demo y ensayo cruzado de defensa: cada uno explica un módulo que no hizo.
8. **Jue 5 nov:** Entrega 2 (colchón: vie 6).

Estructura de carpetas

Es un solo proyecto Maven (no multimódulo) que produce un solo JAR. Se organiza por capas, igual que en la arquitectura de la Entrega 1, y dentro de cada capa por módulo funcional. Los nombres de paquetes siguen los del cronograma (`domain.model`, `domain.port`, `domain.rules`).

```
inventiq/
├── .github/workflows/ci.yml          # pruebas + JaCoCo en cada PR
├── .mvn/wrapper/                     # wrapper de Maven (obligatorio)
├── mvnw  ·  mvnw.cmd
├── pom.xml
├── README.md                         # qué es, cómo levantar, cómo probar,
uso de IA
├── CLAUDE.md                         # reglas para Claude Code
├── ACUERDOS.md
├── docs/
│   ├── entrega1.pdf
│   ├── arquitectura.md               # capas, modelo de dominio,
dependencias
│   └── diagramas/
└── src/
    ├── main/
    │   ├── java/co/edu/unal/inventiq/
    │   │   ├── InventiqApplication.java
    │   │   │
    │   │   ├── domain/                       # Java puro: sin Spring, sin
JPA
    │   │   │   ├── model/                    # Product, Batch, Sale,
SaleItem, Supplier,
    │   │   │   │                             # PurchaseOrder, OrderLine,
OrderStatus, Settings, Alert...
    │   │   │   ├── rules/                    # FifoAllocator,
SalesVelocityCalculator,
    │   │   │   │                             # ReorderPointCalculator,
DraftOrderGrouper
    │   │   │   ├── port/                     # interfaces hacia afuera:
```

```
ProductRepository,
    |   |   |   |                           # BatchRepository,
SaleRepository, MessageNotifier...
    |   |   |   └── exception/              # UnknownBarcodeException,
InsufficientStockException
    |   |   |
    |   |   ├── application/                # casos de uso: orquestan
dominio + puertos
    |   |   |   ├── sales/                  # SalesService
(B)
    |   |   |   ├── replenishment/          # ReplenishmentService
(C)
    |   |   |   ├── purchasing/             # PurchasingService,
ReceiptService (D)
    |   |   |   └── admin/                  # CatalogService,
SettingsService,
    |   |   |                               # AuditRecorder, AlertService
(D)
    |   |   |
    |   |   ├── infrastructure/             # implementa los puertos del
dominio
    |   |   |   ├── persistence/
    |   |   |   |   ├── entity/             # ProductEntity, BatchEntity...
(@Entity)
    |   |   |   |   ├── jpa/                # interfaces Spring Data
    |   |   |   |   ├── mapper/             # Entity <-> modelo de dominio
    |   |   |   |   └── adapter/            # ProductRepositoryAdapter
implements ProductRepository
    |   |   |   ├── notification/           # LogMessageNotifier (E2),
TwilioMessageNotifier (E3)
    |   |   |   ├── security/               # SecurityConfig,
UserDetailsService (A)
    |   |   |   └── config/                 # Clock, beans generales
    |   |   |
    |   |   └── web/                        # @Controller + formularios;
nunca @RestController
    |   |       ├── auth/                   # login, logout
(A)
    |   |       ├── pos/                    # pantalla de ventas
(B)
    |   |       ├── inventory/              # vista de lotes
(B)
    |   |       ├── replenishment/          # pantalla del motor
(C)
    |   |       ├── purchasing/             # pedidos y recepción
(D)
```

```
    |   |       └── admin/                    # productos, proveedores,
parámetros (D)
    |   |
    |   └── resources/
    |       ├── application.properties        # H2 en archivo, ddl-
auto=validate
    |       ├── application-demo.properties   # agrega db/demo a Flyway
    |       ├── db/migration/                 # V1__esquema_inicial.sql, ...
    |       ├── db/demo/                      # datos de demostración
(C)
    |       ├── templates/
    |       |   ├── layout.html               # plantilla base
(D)
    |       |   ├── fragments/                # cabecera, menú, mensajes
    |       |   ├── auth/  pos/  inventory/  replenishment/  purchasing/
admin/
    |       └── static/css/  static/js/       # js mínimo: foco del escáner
    |
    └── test/java/co/edu/unal/inventiq/
        ├── domain/rules/                     # pruebas unitarias puras
(cuentan para el 80%)
        ├── application/                      # servicios probados con fakes,
sin Spring
        ├── fakes/                            # InMemoryProductRepository,
FakeMessageNotifier...
        ├── infrastructure/persistence/       # @DataJpaTest contra H2
        ├── web/                              # @WebMvcTest y pruebas de
seguridad
        └── architecture/                     # ArchUnit: el dominio no
importa Spring ni JPA
```

Las dependencias solo van en un sentido: `web` → `application` → `domain` ← `infrastructure`. El dominio no conoce a nadie y la infraestructura implementa sus puertos.

- **Dos modelos separados:** `Product` del dominio y `ProductEntity` de persistencia. Duplica un poco, pero es lo que permite probar las reglas sin base de datos, y eso se califica.
- **Servicios sin anotaciones en el constructor:** los de `application` reciben los puertos por constructor, así las pruebas los arman a mano con los fakes.
- **Fecha inyectada:** el ROP y los vencimientos dependen de "hoy", así que usen un `Clock` inyectado. Así las pruebas pueden fijar la fecha.

- **Prueba de arquitectura:** una prueba con ArchUnit que falle si `domain` importa `org.springframework` o `jakarta.persistence` convierte el requisito de "lógica aislada" en algo verificable. Va con A3.

Parte A: base, seguridad e infraestructura (Alejandro)

A deja el proyecto listo para que los demás trabajen y es responsable del login (FR-01), del arranque con un solo comando (NFR-05) y del documento de arquitectura. Ya no hace los repositorios de todos ni los datos semilla.

ID	Fechas	Tarea	Listo cuando	Depende de
A1	16–17 oct	Proyecto Spring Boot (Java 21, Maven con wrapper), H2 en archivo, Flyway con V1 que crea todas las tablas de ACUERDOS §3, `ddl-auto=validate`	`./mvnw spring-boot:run` arranca y crea las tablas	Reunión del sáb 17
A2	17–20 oct	Paquete de dominio (entidades y enums) e interfaces de puertos y servicios, en Java puro, sin anotaciones de Spring ni JPA	PR fusionado; B, C y D ramifican desde ahí	A1
A3	19–20 oct	JaCoCo con umbral de 80% sobre el dominio, GitHub Actions en cada PR y protección de `main`	el build falla si baja de 80%; no se puede hacer push a `main`	A1
A4	20 oct	README v1: qué es, cómo levantarlo, cómo correr las pruebas, flujo de ramas y PR, declaración de uso de IA	un compañero arranca el proyecto solo con el README	A1
A5	21–23 oct	Base de persistencia (entidades JPA separadas del dominio y mappers) y repositorios de Product, Batch y User; `findAvailableByProduct` en orden FIFO; `@Version` en Product y Batch	pruebas de repositorio en verde	A2
A6	23 oct	NotificadorMensajes falso que escribe en el log (Twilio queda para la Entrega 3)	implementa el puerto y tiene prueba	A2

ID	Fechas	Tarea	Listo cuando	Depende de
A7	23–27 oct	Login con 2 roles (FR-01): Spring Security, sesión, BCrypt, páginas de entrada y salida, redirección por rol	el cajero solo entra a ventas y recibe 403 en rutas de admin	A5, D1
A8	27–28 oct	Pruebas de seguridad: rutas de admin con cajero y sin sesión; contraseñas guardadas con BCrypt	pruebas en verde en CI	A7
A9	28–29 oct	Empaquetar el JAR y probar `java -jar` en una máquina limpia con solo Java 21	arranca con un solo comando	Integración
A10	30–31 oct	Corregir lo que salga de la integración y etiquetar v0.2	etiqueta creada	Integración
A11	29 oct–3 nov	Documento de arquitectura (capas, diagrama del modelo de dominio, dependencias entre componentes) y armado del documento con las secciones de las 4 letras	documento completo y revisado por otra letra	B10, C9, D8

Nota sobre Flyway: si varias personas agregan migraciones, usen versiones con fecha y hora (`V20261022_1430__...`) para que no choquen dos `V2`.

Parte B: POS e inventario

B entrega la venta por código con deducción FIFO (FR-02, US-03), que es la regla de punta a punta más visible de la demo. Ahora también hace los repositorios de ventas.

ID	Fechas	Tarea	Listo cuando	Depende de
B1	17–19 oct	Repositorios falsos en memoria y pruebas de US-03 escritas primero (venta con 2 lotes, código inexistente)	pruebas escritas y en rojo	Reunión del sáb 17

ID	Fechas	Tarea	Listo cuando	Depende de
B2	19–21 oct	Regla pura `FifoAllocator`: reparte una cantidad entre lotes (vence primero, luego recibido primero), excluye vencidos, una línea por lote	US-03 esc. 1 en verde, más casos de lote insuficiente y lote vencido	B1, A2
B3	21–22 oct	Repositorios de Sale y SaleItem (antes en A6)	pruebas de repositorio en verde	A5
B4	22–23 oct	`SalesService` (iniciar, escanear, cerrar): usa FIFO, cobra el precio de venta y lanza `UnknownBarcodeException` o `InsufficientStockException`	US-03 esc. 2: un código inexistente no cambia stock ni registra venta	B2, B3
B5	22–24 oct	Vista de inventario del admin: stock por producto y lotes con vencimiento y cantidad	el admin ve los lotes en orden FIFO	A5, D1
B6	24–27 oct	Pantalla de ventas: campo con foco automático para el escáner (teclado y Enter), entrada manual siempre visible, líneas, total, errores que no cortan la venta	un cajero registra 10 productos sin usar el mouse	B4, D1, A7
B7	27–28 oct	Concurrencia: reintento ante `OptimisticLockException` y prueba con 2 hilos sobre el mismo lote	nunca queda stock negativo	B4, A5
B8	28–29 oct	Pruebas de integración de ventas con H2 y prueba de aceptación completa de US-03	verde en CI	B6, B7
B9	30–31 oct	Corregir lo que salga de la integración y congelar el módulo	sin errores abiertos del módulo	Integración
B10	2–3 nov	Sección del documento: módulo POS, diagrama de secuencia de una venta, capturas y reporte de pruebas	sección revisada por otra letra	B9

Ojo con US-03: el código de barras no distingue lotes, así que la prueba debe expresar que se descuenta del lote que vence primero, no del lote "que el cajero escaneó".

Parte C: motor de reabastecimiento

C entrega el cálculo del ROP (FR-03) y los borradores por proveedor (FR-04, US-01). Es lo que responde a "¿qué decide el software que hoy decide un humano?". Ahora también arma los datos de demostración, porque sin historial realista el ROP no se puede mostrar.

ID	Fechas	Tarea	Listo cuando	Depende de
C1	17–19 oct	Pruebas escritas primero para velocidad de ventas y ROP (sin historial, ventana de 30 días, plazo de entrega, stock de seguridad)	pruebas escritas y en rojo	Reunión del sáb 17
C2	19–20 oct	`SalesVelocityCalculator`: unidades por día, vacío si no hubo ventas en la ventana; avisar a B cómo usarlo	pruebas en verde y método documentado	C1, A2
C3	20–22 oct	`ReorderPointCalculator` y cantidad sugerida (redondeo hacia arriba, mínimo 1; sin historial usa el stock de seguridad)	pruebas en verde con 90% o más de cobertura en las reglas	C2
C4	22–24 oct	Datos de demostración (perfil `demo`, antes A8): 1 admin, 1 cajero, 4–5 proveedores, 80–100 productos con código de barras, lotes con algunos por vencer, productos sin historial y 60–90 días de ventas simuladas	arrancar con el perfil `demo` deja la BD poblada	A1
C5	24–26 oct	`recalculateReorderPoints()`: lee ventas, stock y plazo del proveedor y guarda el ROP de cada producto	con los datos de demostración se actualiza el ROP de todos los productos	C3, C4, B3, D2
C6	26–28 oct	`generateDraftOrders()` (FR-04, US-01): un borrador por proveedor, uno "por revisar" para productos sin proveedor y sin duplicar borradores abiertos	esc. 1: 4 + 2 productos dan 2 borradores con total; esc. 2: pedido "por revisar" y alerta	C5, D3
C7	28–29 oct	Pantalla "Reabastecimiento" (admin): stock, ROP, velocidad y cantidad sugerida por producto, y botón "Generar borradores"	el admin genera borradores desde la pantalla	C6, D1

ID	Fechas	Tarea	Listo cuando	Depende de
C8	30–31 oct	Primera medición de rendimiento con 1.000 productos sintéticos (NFR-01) y corrección de errores de la integración	tiempo medido y anotado; plan si pasa de 1,5 s	C6, Integración
C9	2–3 nov	Sección del documento: algoritmo del ROP con un ejemplo numérico, pruebas y medición	sección revisada por otra letra	C8

Parte D: compras y administración

D entrega la plantilla base, los parámetros (FR-08), la recepción de pedidos (FR-10, US-05) y el flujo de aprobación de borradores. Ahora también hace los repositorios de proveedores, pedidos, parámetros, auditoría y alertas.

ID	Fechas	Tarea	Listo cuando	Depende de
D1	17–19 oct	Plantilla base Thymeleaf (layout, cabecera, menú por rol, estilos, mensajes) y bocetos de las pantallas de admin	PR fusionado; A, B y C usan el layout	A1
D2	19–21 oct	Repositorios de Supplier, PurchaseOrder, Settings, AuditLog y Alert (antes A6–A7)	pruebas de repositorio en verde	A2
D3	21–23 oct	`AuditRecorder` y `AlertService` con persistencia, y lista de alertas en el inicio del admin	B y C ya pueden registrar alertas y auditoría	D2
D4	22–24 oct	Productos y proveedores: crear, editar e inactivar. Proveedor: nombre, WhatsApp, plazo, MOQ en monto o unidades. Producto: código único, precio, costo, perecedero, proveedor, stock de seguridad	el admin los gestiona; el cajero recibe 403	D1, D2, A5

ID	Fechas	Tarea	Listo cuando	Depende de
D5	24–26 oct	Parámetros (presupuesto semanal, ventana de ventas, WhatsApp del admin) y ajuste manual de stock (FR-08), con auditoría	los cambios quedan como `SETTINGS_CHANGED` y `STOCK_ADJUSTED`	D3, D4
D6	26–28 oct	Recepción de pedidos (FR-10, US-05): un lote por línea con su vencimiento; recepción parcial deja el pedido en `PARTIALLY_RECEIVED` con pendientes visibles	US-05 esc. 1 y 2 en verde; el stock sube	D2, A5
D7	28–30 oct	Pedidos: lista de borradores, edición de cantidades, aprobar (DRAFT a APPROVED) y marcar como enviado a mano, solo con transiciones válidas	el admin edita y aprueba un borrador generado por C	C6, D6
D8	2–3 nov	Sección del documento: administración, pedidos y recepción, con capturas	sección revisada por otra letra	D7

Las transiciones de estado del pedido (DRAFT, APPROVED, SENT, PARTIALLY_RECEIVED, RECEIVED) son lógica de negocio que se califica: deben vivir en el dominio con sus propias pruebas, no en el controlador.

Reglas comunes y criterios de entrega

Todos trabajan igual con Claude Code, para que el historial y la defensa salgan bien.

- Una rama por tarea (`feature/B2-fifo`), commits propios bajo su identidad y PR aprobado por otra letra; nada directo a `main` .
- Claude Code en modo Plan o Manual, nunca en edición automática. Cada uno lee y entiende el diff antes de aceptarlo.
- El `CLAUDE.md` del repo es la fuente de reglas para Claude. Cambiarlo, o cambiar el dominio o los puertos, requiere a los 4.
- Las reglas de negocio llevan pruebas sin Spring ni base de datos; el build falla si el dominio baja de 80%.

- Quien termina algo de lo que depende otra letra avisa ese mismo día. Un atraso de 2 días se avisa sin esperar a la siguiente reunión.

La Entrega 2 está lista cuando se cumplen estas condiciones:

- [ ] El proyecto arranca con un solo comando en una máquina limpia con Java 21.
- [ ] El ciclo login, venta, ROP, borrador, aprobación y recepción corre en la demo sin tocar la BD a mano.
- [ ] Las pruebas pasan en CI y el dominio tiene 80% o más de cobertura.
- [ ] El cajero no puede abrir ninguna pantalla del administrador.
- [ ] El README incluye cómo levantar, cómo probar y la declaración de uso de IA.
- [ ] El documento de arquitectura y las secciones de las 4 letras están completos y revisados.
- [ ] Cada integrante puede explicar al menos un módulo que no escribió.